AI INTERVIEW COACH

Complete Workflow, Data Flow & Implementation Guide

This guide walks through exactly how the app works end-to-end, how data moves through it, how to actually build it step by step, and the reasoning behind every technical choice — so you understand the *why*, not just the *what*.

1. System Workflow — The User Journey

This is the exact path a student's action takes through the app, from opening it to seeing their result. Follow the arrows top to bottom — each box is one thing the system does.

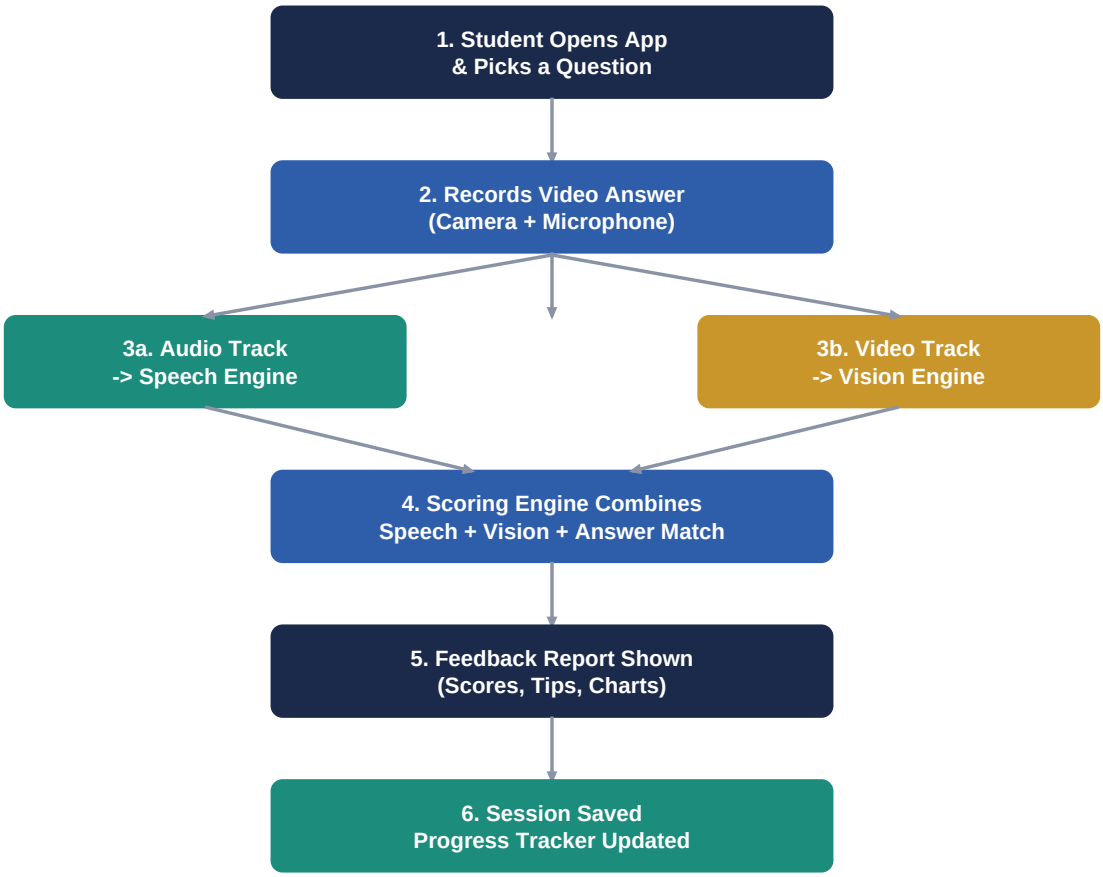

Fig 1.1 — End-to-end workflow of a single mock interview session

Walking Through Each Stage

- **Stage 1 – Question Selection:** The student picks a question type (HR / Technical / Behavioral). This decides which "ideal answer template" will later be used for scoring.
- **Stage 2 – Recording:** CameraX captures video; the audio track is extracted from the same recording so nothing is recorded twice.
- **Stage 3 – Parallel Processing:** This is the key design idea — audio and video are analyzed *at the same time* by two independent engines, not one after another. This is what makes the app fast.
- **Stage 4 – Scoring Engine:** Once both engines finish, their outputs (filler count, pace, eye-contact %, answer relevance) are combined using a weighted formula into one final score.
- **Stage 5 – Report:** The score is turned into a human-readable report with specific, actionable tips — not just a number.
- **Stage 6 – Save & Track:** The session is stored so the next attempt can be compared against it, creating the "progress over time" feature.

2. Data Flow Diagram (DFD)

A workflow shows *actions*. A data flow diagram shows *information* — where it comes from, where it's temporarily held, and where it finally rests. This matters a lot for your SRS and viva, since examiners often ask "where exactly is this data stored?"

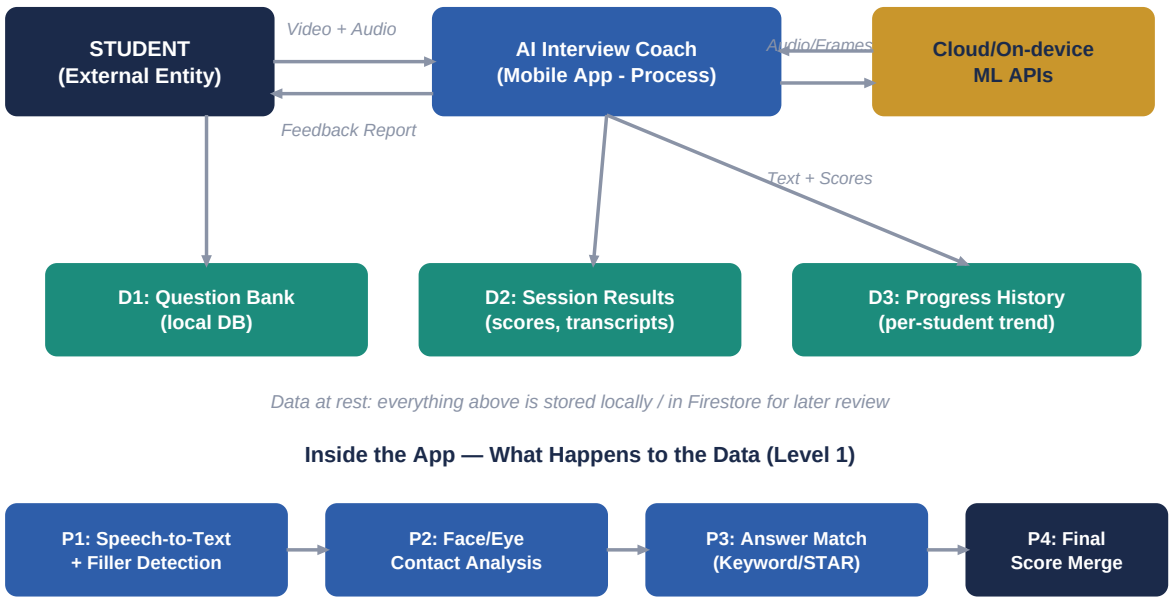

Fig 2.1 — Context-level (top) and Level-1 (bottom) data flow

Reading the Diagram

- **External entities** (dark navy boxes) are outside the system's control — the Student and the Cloud/On-device ML APIs.
- **The process** (blue box) is your app itself — it never stores data permanently, it only transforms it.
- **Data stores** (teal boxes, labeled D1/D2/D3) are where information actually rests: the question bank, session results, and long-term progress history.
- **P1–P4** at the bottom are the four internal mini-processes that do the real work — this is what you'd expand into your Level-2 DFD in the SRS if asked for more depth.

Why This Matters

Notice the app never sends raw video to a permanent server store — only the *extracted signals* (text, scores, landmark points) are saved. This is a genuinely important design decision: it keeps storage costs low and is far more privacy-respecting, since you are not hoarding video recordings of students. Mentioning this explicitly in your report will impress evaluators.

3. Concept Deep-Dive — The "Why" Behind Each Part

Understanding these ideas properly means you can answer any viva question confidently, in your own words, instead of memorizing.

3.1 Speech-to-Text (STT)

Speech-to-text converts spoken audio into written words. Internally, it breaks the audio into tiny time slices (milliseconds), extracts sound-pattern features, and matches those patterns to phonemes (basic speech sounds) using a trained neural network, then stitches phonemes into words using a language model that knows which word sequences are likely. Think of it like predictive text, but for sound instead of typing.

3.2 Filler Word & Pace Detection

Once you have the transcript, this part is surprisingly simple: it's mostly counting. The app scans the text for a known list of filler words ("um", "like", "you know") and counts how many words were spoken divided by the recording's duration to get words-per-minute (pace). No heavy ML is needed here — this is a good place to save development time.

3.3 Face & Eye-Contact Analysis (Computer Vision)

ML Kit's face detector finds key landmark points on the face (eyes, nose, mouth corners) in every video frame. By tracking whether the eyes are pointed roughly toward the camera across frames, the app calculates what percentage of the time the student maintained eye contact. This runs entirely on the phone ("on-device"), which means it works without internet and doesn't send anyone's face to a server.

3.4 Answer Quality Scoring (STAR Method Matching)

For behavioral questions, good answers usually follow the STAR structure: Situation, Task, Action, Result. The app doesn't need advanced AI to check this — a simple keyword/pattern approach works: does the transcript mention a specific situation, describe an action taken, and state a result/outcome? Each part found adds to the score. This is explainable and easy to defend in a viva, unlike a black-box ML model.

3.5 Combining Scores (Weighted Formula)

The final score is not one AI model deciding everything — it's a transparent formula, e.g. Final Score = 30% Speech Clarity + 25% Eye Contact + 25% Answer Structure + 20% Pace. Using simple, explainable weights (rather than a single opaque ML score) is both easier to build correctly in one semester and easier to justify to your evaluators.

3.6 On-device vs Cloud Processing

On-device (ML Kit face detection) means the phone does the work itself: faster, private, free, but limited in power. Cloud (speech-to-text APIs) means sending data to a server that does heavier processing: more accurate, but needs internet and may cost money at scale. Using both, as this project does, is a realistic and practical trade-off you should be able to explain.

4. Implementation Tips — Building It Without Getting Stuck

Build Order (Do Not Skip This)

- Get a plain Android app recording video and saving it locally **first**, before touching any ML. A working camera screen is your foundation.
- Add the question bank next (simple local JSON or SQLite list) — this gives you something to demo very early.
- Integrate speech-to-text third. Test it on 5–10 sample recordings before trusting it.
- Add face/eye detection fourth, as a separate independent module — keep it decoupled from speech code so one team member can build each in parallel.
- Build the scoring formula and report screen last, once you actually have real numbers coming from both engines to design the UI around.

Team Split Suggestion (3–4 members)

- **Member A:** Android UI/UX – recording screen, question bank, navigation.
- **Member B:** Speech pipeline – STT integration, filler/pace logic.
- **Member C:** Vision pipeline – face detection, eye-contact calculation.
- **Member D:** Backend/storage – Firebase setup, scoring formula, report + charts screen.

Practical Gotchas to Watch For

- Speech-to-text APIs often have a free-tier limit – test with short clips (under 30 seconds) to avoid burning quota during development.
- Face detection accuracy drops in poor lighting – mention this as a known limitation in your report rather than hiding it; evaluators respect honesty about constraints.
- Video files are large – do not upload full video to Firestore (it's a document DB, not built for large blobs). Use Firebase Storage for the file, Firestore only for the extracted results.
- Always test on a real device, not only an emulator – camera and microphone behavior differs significantly.

Testing Strategy

- Unit test the filler-word counter and scoring formula separately – these are pure logic and easiest to get marks for correctness.
- For ML components, prepare 3–4 known sample videos (one excellent answer, one nervous/filler-heavy answer) so your demo always produces a clearly good vs. clearly weak result – this makes the app's value obvious to evaluators.

5. Suggested Android Project Structure

A clean folder structure makes it far easier for four people to work without constantly overwriting each other's code, and it looks professional in your final report.

```
app/
 |-- ui/
 |    |-- recording/        (camera screen, record button)
 |    |-- questionbank/     (question list, filters)
 |    |-- report/           (score screen, charts)
 |    `-- history/          (progress tracker screen)
 |-- speech/
 |    |-- SpeechToTextEngine.kt
 |    |-- FillerWordDetector.kt
 |    `-- PaceCalculator.kt
 |-- vision/
 |    |-- FaceAnalyzer.kt
 |    `-- EyeContactCalculator.kt
 |-- scoring/
 |    |-- AnswerMatcher.kt    (STAR method check)
 |    `-- ScoreAggregator.kt  (final weighted formula)
 |-- data/
 |    |-- local/             (Room/SQLite - question bank)
 |    `-- remote/            (Firebase - session + progress)
 `-- model/                  (data classes: Session, Question, Report)
```

6. Quick Concept Cheat-Sheet

Term	In One Line
STT (Speech-to-Text)	Turns spoken audio into written text using trained sound-pattern models.
ML Kit	Google's free, on-device machine learning toolkit — used here for face detection.
DFD	Data Flow Diagram — shows how information moves and where it is stored, not what actions happen.
STAR Method	Situation, Task, Action, Result — the ideal structure for a behavioral interview answer.
On-device ML	Processing that runs on the phone itself — private, fast, offline, but less powerful.
Cloud ML	Processing that runs on a remote server — more powerful, but needs internet and may cost money.
Weighted Scoring	Combining several scores using fixed percentages instead of one black-box model.
Firebase Firestore	A cloud database good for small structured data — not for storing large video files.